

2D Geometric Transformation

Basic transformations :

Translation



Introduction

Translation is one of the fundamental operations in **2D geometric transformations** used in computer graphics, where an **object is moved from one position to another** without changing its shape, size, or orientation.

It is the simplest transformation and is widely used in **animation, gaming, and graphical applications to reposition objects on the screen.**

◆ Definition

Translation is the process of shifting every point of an object by a fixed distance in a given direction.

👉 In simple words:

It means **moving an object without rotating or resizing it.**

◆ Mathematical Representation

If a point **P(x, y)** is translated by:

- **tx** units along x-axis
- **ty** units along y-axis

Then the new point **P'(x', y')** is:

$$x' = x + t_x, \quad y' = y + t_y$$

◆ Matrix Representation of Translation

Translation can also be represented using **homogeneous coordinates**:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

👉 This matrix form is important because:

- It allows combining multiple transformations
 - It is used in **computer graphics pipelines**
-

◆ How Translation Works

1. Each point of the object is taken individually
2. A translation vector **(tx, ty)** is added
3. All points move in the same direction
4. The shape remains exactly the same

👉 Example:

If (2, 3) is moved by (4, 1),

New point = (6, 4)

◆ Translation Vector

The pair **(tx, ty)** is called the **translation vector**.

- Positive tx → move right
 - Negative tx → move left
 - Positive ty → move up
 - Negative ty → move down
-

◆ Properties of Translation

- ✓ Shape and size remain unchanged
 - ✓ Angles remain unchanged
 - ✓ Parallel lines remain parallel
 - ✓ It is a **rigid body transformation**
 - ✓ No distortion occurs
-

◆ Applications of Translation

- Moving objects in animations 🎮
 - Scrolling images on screens
 - Positioning UI elements
 - Game character movement
 - Simulation and modeling
-

◆ Advantages

- Simple and easy to implement
 - Requires less computation
 - Maintains object integrity
-

◆ Limitations

- Cannot rotate or resize objects
 - Only changes position
-

◆ One-Line Conclusion

Translation is a basic 2D transformation that shifts an object from one position to another by adding a fixed displacement to all its points without altering its shape or orientation.

Introduction

Rotation is a fundamental 2D geometric transformation in computer graphics in which an object is turned about a fixed point (called the pivot point) through a specified angle, without changing its shape or size.

It is widely used in animations, simulations, robotics, and graphical modeling to change the orientation of objects.

◆ Definition

Rotation is the process of moving every point of an object along a circular path around a fixed point by a certain angle.

👉 In simple words:

It means spinning an object around a point.

◆ Mathematical Representation (About Origin)

If a point P(x, y) is rotated by an angle θ (theta) about the origin (0,0), the new coordinates P'(x', y') are:

$$x' = x \cos \theta - y \sin \theta, \quad y' = x \sin \theta + y \cos \theta$$

👉 Here:

θ is measured in anticlockwise direction (positive rotation)

Clockwise rotation is taken as negative angle

◆ Matrix Representation

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

👉 This matrix form is used in:

Graphics engines

Transform pipelines

Combining transformations

◆ Rotation About an Arbitrary Point (Very Important)

If rotation is about a point (h, k) instead of origin:

Steps:

Translate the point to origin

Apply rotation

Translate back

👉 Formula:

$$x' = h + (x - h)\cos\theta - (y - k)\sin\theta$$

$$y' = k + (x - h)\sin\theta + (y - k)\cos\theta$$

◆ How Rotation Works

Each point of the object is considered

It is rotated using sine and cosine values

All points move along circular paths

The entire object rotates as one unit

◆ Direction of Rotation

↺ Anticlockwise → Positive angle

↻ Clockwise → Negative angle

◆ Properties of Rotation

✓ Shape and size remain unchanged

✓ Distances between points remain constant

✓ It is a rigid body transformation

✓ Orientation changes

✓ Angles are preserved

◆ Example

Rotate point (1, 0) by 90° anticlockwise:

$$X' = 0, y' = 1$$

👉 So, (1,0) becomes (0,1)

◆ Applications

Animation and game development 🎮

Rotating objects in simulations

Robotics movement

Image processing

CAD systems

◆ Advantages

Maintains object structure

Smooth transformation

Essential for realistic graphics

◆ Limitations

Requires trigonometric calculations

Slightly complex compared to translation

◆ One-Line Conclusion

Rotation is a 2D transformation that turns an object around a fixed point by a specified angle while preserving its shape and size.

Scaling

Introduction

Scaling is a fundamental 2D geometric transformation in computer graphics in which the size of an object is increased or decreased with respect to a reference point, without changing its basic shape.

It is used in zooming, resizing images, and adjusting object dimensions in graphical systems

◆ Definition

Scaling is the process of modifying the size of an object by multiplying its coordinates by scaling factors.

👉 In simple words:

It means enlarging or shrinking an object.

◆ Mathematical Representation

If a point P(x, y) is scaled by:

S_x along x-axis

S_y along y-axis

Then the new point P'(x', y') is:

$$x' = x \cdot S_x, \quad y' = y \cdot S_y$$

◆ Matrix Representation

$$\begin{bmatrix} x' & y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \begin{bmatrix} x & y \end{bmatrix}$$

👉 This matrix is used in:

Graphics pipelines

Transformation combinations

Animation systems

◆ Types of Scaling

✓ 1. Uniform Scaling

$$S_x = S_y$$

Object size changes proportionally

Shape remains exactly the same

👉 Example: Enlarging a square into a bigger square

✓ 2. Non-Uniform Scaling

$S_x \neq S_y$

Object is stretched or compressed

Shape may distort

👉 Example: Rectangle becoming wider but not taller

◆ Scaling About Origin

Object expands or shrinks relative to (0,0)

Points move away from or toward the origin

◆ Scaling About a Fixed Point (Very Important)

If scaling is done about a point (h, k):

Steps:

Translate object to origin

Apply scaling

Translate back

👉 Formula:

$$x' = h + (x - h)S_x$$

$$y' = k + (y - k)S_y$$

◆ How Scaling Works

Each point of the object is taken

Coordinates are multiplied by scaling factors

All points move proportionally

Object size changes accordingly

◆ Effect of Scaling Factors

$S > 1 \rightarrow$ Enlargement

$0 < S < 1 \rightarrow$ Reduction

$S = 1 \rightarrow$ No change

$S < 0 \rightarrow$ Reflection + scaling

◆ Properties of Scaling

✓ Changes size of object

✓ May change shape (if non-uniform)

✓ Does not preserve distances

✓ Parallel lines remain parallel

◆ Example

Scale point (2, 3) with $S_x = 2$ and $S_y = 3$:

$$x' = 4, y' = 9$$

👉 New point = (4, 9)

◆ Applications

Zooming in/out 🔍

Image resizing

Game object resizing 🎮

CAD design

UI scaling

◆ Advantages

Simple to implement

Useful for resizing

Flexible (uniform & non-uniform)

◆ Limitations

Can distort shape

Does not preserve proportions (in non-uniform scaling)

◆ One-Line Conclusion

Scaling is a 2D transformation that changes the size of an object by multiplying its coordinates with scaling factors along the x and y directions